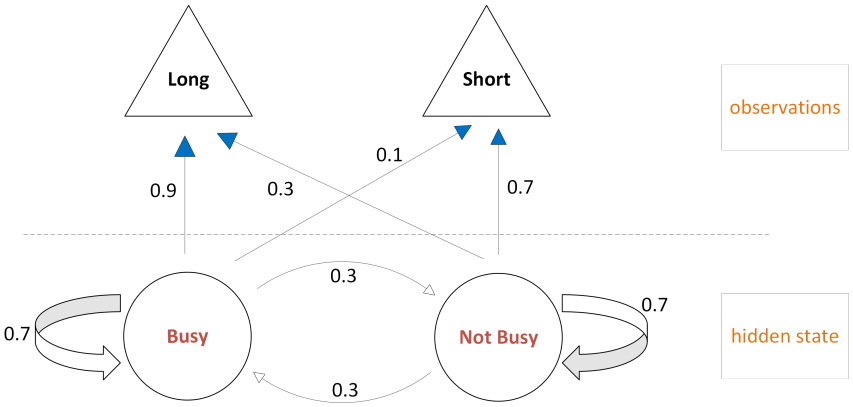
Chapter 4: Hidden Markov Models

Honours Time Series Module

October 2026

Introduction

Suppose that your internet connection is broken and you need to phone your service provider to come and fix it. Suppose that the service provider can be in one of two states: busy or not busy. Depending on this state (which you don't know), your waiting time will be longer or shorter than 3 minutes (on the phone) – two possible outcomes.



It is likely that the state of the service provider doesn't randomly change between busy and not, but has some degree of persistence in time, and that there are phases when they are busy and phases when they are not busy.

Hidden Markov models (HMMs) are models in which observations (and their generating distributions) depend on the state of an underlying and unobserved Markov process (Zucchini and MacDonald (2009)). The states are discrete, i.e., HMMs are state space models with discrete states. Again the state is *unobserved*, and one objective is to estimate the state at time t given the observations.

The hidden states in a Hidden Markov model are modelled as a first-order Markov process (Figure 1), with a discrete state space. Given

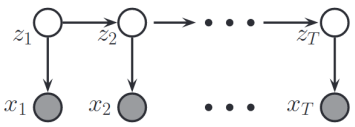


Figure 1: DAG (directed acyclic graph) for a Hidden Markov model: z_t is the hidden state at time t, x_t is the observation at time t.

different possible states that could have generated an observation, the marginal (unconditional) distribution of each observation is a mixture of distributions. Therefore, before giving more details on Hidden Markov models, these notes start with an introduction to mixture models, and Markov processes.

Mixture Models

In a mixture model observations are modelled as coming from a mixture of several distributions. For any particular observation it is unknown from which of these distributions it came. For example, if we have one fair and one loaded dice, and don't know which one was used, a six could have come from either distribution and we would model this using a mixture of two multinomial distributions.

If we have a mixture of K distributions, then:

$$p(x_i) = \sum_{k=1}^{K} \pi_k p_k(x_i)$$

where $p_k(x_i) = p(x_i|z_i = k)$, i.e., the probability of observing x_i from distribution k. The mixing weights π_k satisfy $0 \leq \pi_k \leq 1$ and $\sum_{k=1}^{K} \pi_k = 1$. K is the number of distributions in the mixture. $p_k(x_i)$ denotes the probability of observing x_i given the k-th distribution. We don't know from which distribution x_i was generated.

We could also see this as a system which is in one of K possible (discrete, latent, unknown) states $z_i \in 1, \ldots, k$. Then, conditional on the state being k, an observation from state-distribution p_k is generated.

Because of the unknown latent state, a mixture model is a form of latent (hidden) variable model.

Figure 2 shows an example of a mixture distribution resulting from the mixture of three Poisson distributions. Given the observed data, the aim will be to identify the parameters of the latent distributions, and their relative proportions in the mixture, and sometimes also the number of underlying latent distributions. Mixture models are also useful for capturing overdispersion or non-homogeneity in a population.

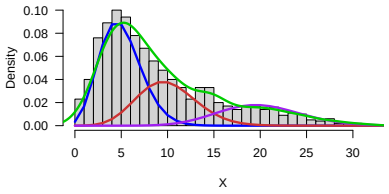


Figure 2: Observations generated from a mixture of three Poisson distributions in green. The latent component distributions are shown in blue, brown and purple.

In the mixture model we have a random sample from the mixture distribution, i.e., the observations are independent.

The likelihood for a two-component mixture model can be written as

$$L(\theta, \psi) = \prod_{i=1}^{n} [\pi p_1(x_i) + (1-\pi) p_2(x_i)]$$
$$= \prod_{i=1}^{n} \left(\sum_{k=1}^{2} \pi_k p_k(x_i) \right)$$

The above is an *independent mixture*. In contrast, in a Hidden Markov model the observations are ordered sequentially, and the underlying state variable follows a Markov process (not independent).

Markov Chains

A Markov chain is a model for a sequence of random outcomes, where the next outcome depends only on the current outcome and, conditional on the current outcome, is independent on earlier outcomes. In other words: A Markov chain is a stochastic process with Markov property.

Markov property:

$$P(X_t | X_{t-1}, \ldots, X_1) = P(X_t | X_{t-1})$$

In a Markov chain, X_t captures all the relevant information for predicting the future, it is a sufficient statistic. A directed acyclic graph (DAG) is a useful tool for illustrating dependence. The DAG for a first-order Markov model is shown in Figure 3. Independent observations would correspond to a graph without links.

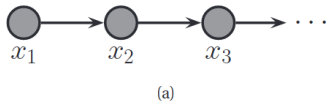


Figure 3: DAG for a first order Markov model.

For a first-order Markov model the joint distribution of all observations can be written as a product of conditional distributions or probabilities.

$$p(\mathbf{x}) = p(x_1) p(x_2|x_1) p(x_3|x_2) \ldots$$
$$= p(x_1) \prod_{t=2}^{T} p(x_t|x_{t-1}).$$

If x_t is a state variable, then the range of possible values for x_t is called the state space. In this chapter we will deal only with Markov processes which are measured at discrete (and equally spaced) time points (discrete-time Markov chain).

If the random variables x_t in a Markov chain are discrete, i.e., $x_i \in \{1, \ldots, K\}$, then this is a discrete-state or finite-state Markov chain; and $p(X_t|X_{t-1})$ can be written as a $K \times K$ transition probability matrix:

$$\Gamma_{ij} = P(X_t = j | X_{t-1} = i) = \gamma_{ij}$$

The rows of matrix Γ sum to 1: $\sum_j \Gamma_{ij} = 1$.

If we assume that the transition function $p(X_t|X_{t-1})$ (or transition probability matrix) is constant over time, then the chain is called homogeneous, stationary or time-invariant.

Stationary distribution: the data evolves in time, however the distribution from which it is generated remains the same. Nonstationary: generative distribution evolves over time.

A first-order Markov model may not be adequate if observations, or trends apart from the previous value contain information, e.g. if the next observation depends on the current trend in the sequence.

We can define a second-order Markov model as

$$\begin{aligned} p(\mathbf{x}) &= p(x_1, x_2)p(x_3|x_2, x_1)\dots \\ &= p(x_1, x_2)\prod_{t=3}^{T} p(x_t|x_{t-1}, x_{t-2}). \end{aligned}$$

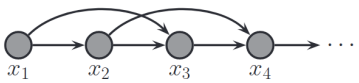


Figure 4: Second-order Markov model.

A first-order Markov model has $K(K-1)$ parameters (transition probabilities). A second-order model has $K^{\ell}(K-1)$ parameters, where ℓ is the order of the Markov chain (Zucchini and MacDonald (2009)). A first-order Markov model may be inadequate if long-range correlations are present. On the other hand, higher-order Markov chains have too many parameters.

Introducing a hidden state through an underlying hidden process, with one latent variable for each observation, gives rise to a graphical structure known as a state space model.

If both latent and observed variables are Gaussian, and all dependencies can be described as linear functions, then we have a dynamic linear model (see Chapter State Space Models).

Hidden Markov Model

In a hidden Markov model, the underlying hidden process follows a first-order Markov chain, with discrete state-space. The data are noisy observations related to the current state. 'Time' could also refer to location in a genomic sequence or text.

Let z_t denote the hidden variable at time t, and x_t the observation at time t. We can write the Hidden Markov model in state space form:

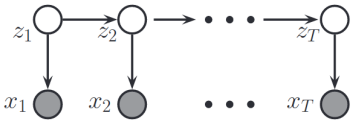
Transition model: $p(z_t|z_{t-1})$

Observation model: $p(x_t|z_t)$

using conditional probability distributions. Observations can be discrete or continuous. Observations are correlated, but conditional on the current state they are uncorrelated. Introduction of a correlated hidden state allows us to model the correlation in the observations indirectly. The hidden variables are sometimes also called *latent* variables.

For example, we can use tree ring data to learn about historic temperatures, if we are willing to classify temperature into states such as cold, medium, warm. Temperature affects tree ring growth, but this information is latent in the tree ring data.

The model has observed random variables (x_t) and latent / hidden / unobserved random variables z_t. The joint distribution of all of these (states and observations) is



$$\begin{aligned} p(\mathbf{z}, \mathbf{x}) &= p(\mathbf{z})p(\mathbf{x}|\mathbf{z}) \\ &= \left[p(z_1)\prod_{t=2}^{T} p(z_t|z_{t-1})\right]\left[\prod_{t=1}^{T} p(x_t|z_t)\right]. \end{aligned}$$

Given the state at time t, the observations $x_t|z_t$ are independent. If the observations are discrete, we can write the observation model as an *observation probability matrix* (usually states along the rows, observations along the columns).

$$B_{kl} = p(x_t = l|z_t = k).$$

An example of data generated from a model with three bivariate Gaussian state-distributions (continuous observations) is shown in Figure 5. The model is

$$[x_t|z_t = k] \sim N(\mu_k, \Sigma_k)$$

with transition probability matrix

$$\Gamma = \begin{pmatrix} 0.8 & 0.1 & 0.1 \\ 0.1 & 0.8 & 0.1 \\ 0.1 & 0.1 & 0.8 \end{pmatrix}$$

HMMs are more efficient in representing long-range dependencies between observations than Markov models (which explicitly model the correlation in the visible sequence), through the latent state process. Note, however, that the Markov property does not hold for the *observations* (only for the states).

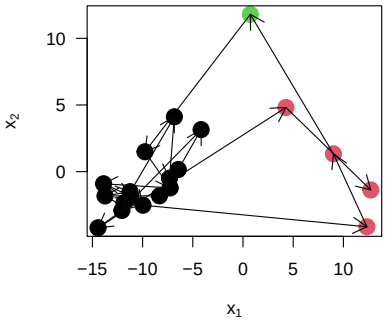


Figure 5: Simulated observations from a three-state HMM with bivariate Gaussian state distributions.

The aims of Hidden Markov models include:

- parameter estimation (transition probabilities, and state distribution parameters)
- point and interval forecasting
- decoding (estimating the sequence of hidden states): $p(z_t|\mathbf{x}, \theta)$

It is common to try and ascribe some meaning to the hidden states, e.g. given observed animal movement, decide whether the animal was in travelling or feeding mode, or given observed tree ring data, infer something about the temperature conditions in a particular year (classify into states).

Applications

HMMs are a common topic in machine learning courses. Areas of application include:

- Speech recognition: the hidden variable is the spoken word, the observations are measurements on acoustic frequencies
- Activity recognition:

  x_t = features from video

  z_t = class of activity (running, walking ...)
- Bioinformatics, Gene finding

  x_t = DNA nucleotides ACGT

  z_t = gene coding region or not
- Changes in regimes: financial markets, El Nino vs La Nina
- Animal movement modelling
- Environmental data: modelling of rainfall

Example: Occasionally dishonest casino

In this example we have two dice. The state variable z_t is the identity of the die, fair ($z = 1$), loaded ($z = 2$), the observation is the number rolled $x_t \in \{1, 2, \ldots, 6\}$.

$(x_t|z = 1) \sim$ uniform multinoulli

$(x_t|z = 2) \sim$ multinoulli skewed towards 6

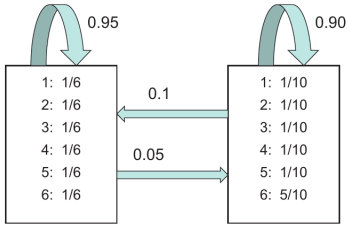


Figure 6: The occasionally dishonest casino. The blue arrows show the state transition probabilities.

Example: Earthquakes

Annual counts of major earthquakes (magnitude 7 and above) (Zucchini and MacDonald (2009)).

From Figure 7 it might seem that there are periods with higher activity, medium activity and lower activity, and that there is correlation in these states.

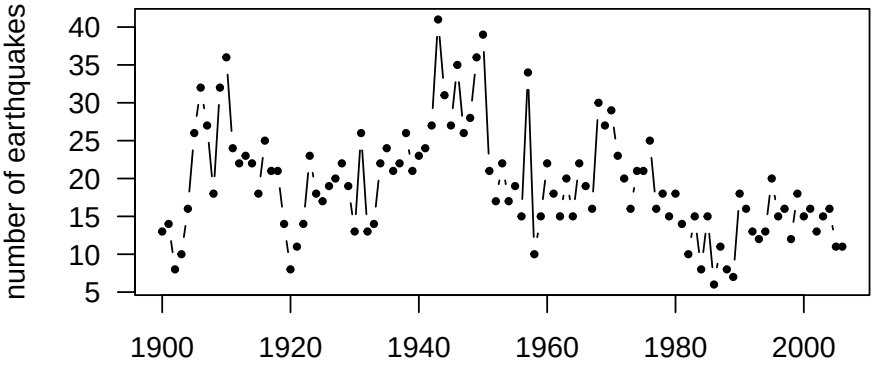


Figure 7: Number of major (magnitude 7 or above) earthquakes per year.

Likelihood

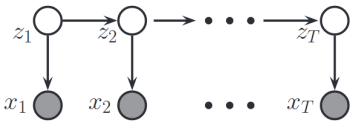


Figure 8: DAG for HMM.

In this section we define the model in more detail. For this we will again refer to the DAG (Figure 8).

The model has observed random variables (x_t) and latent / hidden / unobserved random variables. The joint distribution of all of these (states and observations) is

$$\begin{aligned} p(\mathbf{z}, \mathbf{x}) &= p(\mathbf{z})p(\mathbf{x}|\mathbf{z}) \\ &= \left[p(z_1)\prod_{t=2}^{T} p(z_t|z_{t-1})\right]\left[\prod_{t=1}^{T} p(x_t|z_t)\right]. \end{aligned}$$

or

$$p(\boldsymbol{x}, \boldsymbol{z}|\theta) = p(z_1|\pi)\prod_{n=2}^{N} p(z_n|z_{n-1}, A)\prod_{m=1}^{N} p(x_m|z_m, \phi)$$

where $\theta = \{\pi, A, \phi\}$ is the set of parameters governing the model.

The likelihood function (for the observed data $\mathbf{x} = \{x_1, \ldots, x_T\}$ only) is obtained from the joint distribution by marginalizing over the latent variable (sum over latent variable). I am changing notation here (C_t is the state at time t).

In any directed graphical model, the joint distribution of a set of random variables V_i is given by

$$Pr(V_1, V_2, \ldots, V_n) = \prod_{i=1}^{n} Pr(V_i|pa(V_i))$$

where $pa(V_i)$ denotes all the parents of V_i.

For a single observation:

$$\begin{aligned} Pr(X_t = x) &= \sum_{i=1}^{m} Pr(C_t = i)Pr(X_t = x|C_t = i) \\ &= \sum_{i=1}^{m} u_i(t)p_i(x) \end{aligned}$$

The first line corresponds to the probability / likelihood for a single observation from a mixture.

$u_i(t)$ denotes the probability that the state equals i at time t, $p_i(x)$ denotes the probability of observing x when the system is in state i. m is the number of states.

This can be written in matrix form:

Law of total probability:

$p(A) = \sum_{k=1}^{K} p(A|B_k)p(B_k) = \sum P(A, B_k)$

$$
\begin{aligned}
Pr(X_t = x) &= (u_1(t), \ldots, u_m(t)) \begin{pmatrix} p_1(x) & & 0 \\ & \ddots & \\ 0 & & p_m(x) \end{pmatrix} \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} \\
&= \mathbf{u}(t)\mathbf{P}(x)\mathbf{1}'.
\end{aligned}
$$

$Pr(X_t = x)$ is the marginal probability for the single observation at time t. To obtain the probability for two observations we need to start with

$$Pr(X_t, X_{t+k}, C_t, C_{t+k}) = Pr(C_t)Pr(X_t|C_t)Pr(C_{t+k}|C_t)Pr(X_{t+k}|C_{t+k})$$

We can find the RHS using the directed graph (joint = product of conditional probabilities, conditioning on parents). Marginalizing, we obtain

$$
\begin{aligned}
Pr(X_t = v, X_{t+k} = w) &= \sum_{i=1}^{m}\sum_{j=1}^{m} Pr(X_t = v, X_{t+k} = w, C_t = i, C_{t+k} = j) \\
&= \sum\sum Pr(C_t = i)p_i(v)Pr(C_{t+k} = j|C_t = i)p_j(w) \qquad (1) \\
&= \sum\sum u_i(t)p_i(v)\gamma_{ij}(k)p_j(w) \\
&= \mathbf{u}(t)\mathbf{P}(v)\Gamma^k\mathbf{P}(w)\mathbf{1}'.
\end{aligned}
$$

The double sum in the first line comes from integrating over all possible values of C_t and all possible values of C_{t+k}.

$u_i(t) = Pr(C_t = i)$

$p_i(x) = Pr(X_t = x|C_t = i)$

$\gamma_{ij}(k)$ denotes the (transition) probability of the state moving from i to j in k steps.

For three observations we have

$$Pr(X_t = v, X_{t+k} = w, X_{t+k+l} = z) = \delta\mathbf{P}(v)\Gamma^k\mathbf{P}(w)\Gamma^l\mathbf{P}(z)\mathbf{1}'.$$

And the **likelihood** can be written as

$$L(\theta) = \delta \mathbf{P}(x_1)\Gamma\mathbf{P}(x_2)\Gamma\mathbf{P}(x_3)\ldots\Gamma\mathbf{P}(x_T)\mathbf{1}'.$$

The parameters in this likelihood are:

1. The transition probabilities ($m \times (m-1)$, constraints ensure that the rows sum to 1).
2. The parameters of the m state distributions $m \times p$.

Γ is the $m \times m$ transition probability matrix.

δ is the initial distribution for the states (m-vector).

$\mathbf{1} = (1, 1, \ldots, 1)$ is a row vector.

Calculating the likelihood involves the product of a large number of small probabilities and quickly leads to numerical computing problems (underflow). Usually we deal with this by calculating the log-likelihood instead. However, in the HMM likelihood, because we need to sum out the latent variables to obtain the likelihood (equation 1), the likelihood is not a product of probabilities of individual observations, but a sum, and therefore it is not directly possible to convert this to a sum of logs.

We can however use the following scaling method/algorithm to find the log-likelihood (Zucchini and MacDonald 2009 pg. 47).

Set $\phi_0 = \delta$ and $\ell = 0$. For $t = 1, ...T$

- $v = \phi_{t-1}\Gamma P(x_t)$
- $u = v\mathbf{1}'$
- $\ell = \ell + \log u$
- $\phi_t = v/u$

return ℓ (the log-likelihood).

Example: 2-state Poisson HMM

The following code estimates the Poisson means assuming a two-state Poisson HMM with observations 2, 8, 6, 3, 6, 1, 0, 0, 4, 7, and transition probabiltiy matrix

$$\Gamma = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$$

```r
init = c(0.5, 0.5)
tpm <- matrix(c(0.9, 0.1, 0.2, 0.8), ncol = 2, byrow = TRUE)
lambdas <- c(1, 5)
x <- c(2, 8, 6, 3, 6, 1, 0, 0, 4, 7)

pois2.hmm.ll <- function (lambda, x, init) {
  n <- length(x)
  phi <- init
  ll <- 0

  for (i in 1:n) {
    Pi <- diag(dpois(x[i], lambda = lambda))
    v <- phi %*% tpm %*% Pi
    u <- v %*% c(1, 1)
    ll <- ll + log(u)
    phi <- v / c(u)
  }

  return(-ll)
}

pois2.hmm.ll(lambda = lambdas, x, init = init)
```

```
         [,1]
[1,] 23.55145
```

```r
(out <- nlm(pois2.hmm.ll, p = c(2, 3), x = x, init = init))
```

```
$minimum
[1] 22.93569

$estimate
[1] 0.3634708 5.1158935

$gradient
[1] -6.998846e-07  2.145839e-07

$code
[1] 1

$iterations
[1] 10
```

The two Poisson rate parameters (lambda) are estimated as 0.36 and 5.11, respectively.

Forecasting, Decoding, State Prediction

Much of inference in HMMs involves learning about the hidden state sequences (once parameters are known).

The terminology used is slightly different to that in state space models, because of the discrete nature of the states, and the importance of knowing the particular state.

Prediction / Forecasting

There are two things we might want to forecast or predict: the state at time $t+h$ and the observation (in the form of a predictive or forecast distribution) at time $t+h$.

For predicting the state at time $t+h$ we require

$$p(C_{t+h} = i | x_{1:t})$$

where $h > 0$ is called the prediction / forecast horizon.

For example, if $h = 2$ we need to find:

$$p(C_{t+2}|x_{1:t}) = \sum_{C_{t+1}} \sum_{C_t} p(C_{t+2}|C_{t+1}) p(C_{t+1}|C_t) p(C_t|x_{1:t})$$

We need the filtered state estimate at time t, and then multiply this by the transition probability matrix to the power of h. The result is a k-vector of probabilities: one probability for being in each of the k states at time $t+h$, given the past observations.

C_t refers to the state at time t, x_t to the observation at time t, and $x_{1:t}$ to all observations up till time t.

This vector of probabilities can be converted to a prediction about future observations using

$$p(x_{t+h}|x_{1:t}) = \sum_{C_{t+h}} p(x_{t+h}|C_{t+h}) p(C_{t+h}|x_{1:t})$$

This defines a predictive density or forecast distribution, which can be used for forecasting observations, and prediction intervals. This predictive density is a mixture of densities, with mixture probabilities defined by the probabilities of being in each of the states at time $t+h$.

Define the n-step transition probability matrix as $\boldsymbol{A}(n)$, with $A_{ij}(n) = P(Z_{t+n} = j | Z_t = i)$, i.e. the probability of transitioning from i to j in n steps. The **Chapman-Kolmogorov equations** state: $A_{ij}(m+n) = \sum_{k=1}^{K} A_{ik}(m) A_{kj}(n)$ i.e., the probability of getting from i to j in $m+n$ steps equals the probability of getting from i to k in m steps and k to j in n steps, summed over all k. Then $\boldsymbol{A}(m+n) = \boldsymbol{A}(m)\boldsymbol{A}(n)$ and $\boldsymbol{A}(n) = \boldsymbol{A}^n$.

Local and Global Decoding

The estimation of the most likely state at time t given observations is called local decoding. For this we require the state probabilities

$$p(C_t = i|x_{1:T})$$

and the state for which this probability is maximum is chosen as the most likely state.

In global decoding we find the most likely *sequence* of states and for this require

$$p(C_{1:T}|x_{1:T})$$

The Viterbi algorithm is an algorithm which can find this most likely sequence in a very efficient way.

Before we go into more detail about forecasting and decoding, we will define *forward and backward probabilities* which will help to define and estimate the above probabilities required for forecasting and decoding.

Forward and Backward Probabilities

Define

$$\begin{aligned}\boldsymbol{\alpha}_t &= \boldsymbol{\delta}\boldsymbol{P}(x_1)\boldsymbol{\Gamma}...\boldsymbol{\Gamma}\boldsymbol{P}(x_t) \\ &= \boldsymbol{\delta}\boldsymbol{P}(x_1)\prod_{s=2}^{t}\boldsymbol{\Gamma}\boldsymbol{P}(x_s)\end{aligned}$$

For $t \geq 2$:

$$\alpha_t = \alpha_{t-1}\Gamma P(x_t)$$

The elements of α_t are called the forward probabilities.

$$\alpha_t(j) = P(x_1, \ldots, x_t, z_t = j)$$

$\alpha_t(j)$ represents the joint probability of observing all of the given data up to time t and being in state $z_t = j$ at time t.

The vector α_t gives this probability for each state $j = 1, \ldots, m$.

We can calculate the likelihood from the forward probabilities:

$$L_T = \alpha_T \mathbf{1}'.$$

The backward probabilities are defined as

$$\beta_t(i) = P(X_{t+1:T} = x_{t+1:T} | C_t = i)$$

$$\boldsymbol{\beta}'_t = \mathbf{\Gamma} \boldsymbol{P}(x_{t+1}) \mathbf{\Gamma} \boldsymbol{P}(x_{t+2}) \dots \mathbf{\Gamma} \boldsymbol{P}(x_T) \mathbf{1}'$$

$\beta_t(j)$ represents the conditional probability for the observations at $t+1$ up to T, given that the state at time t is j ($C_t = j$).

The product

$$\alpha_t(j)\beta_t(j) = P(X^{(T)} = x^{(T)}, c_t = j)$$

and, for any t:

$X^{(T)} = X_1, ...X_T$

$$\alpha_t \beta'_t = P(X^{(T)} = x^{(T)}) = L_T$$

State Prediction

As for state space models we refer to filtering, smoothing and forecasting, depending on where the state to be predicted is relative to the observations.

Remember that the likelihood $L_T = P(X^{(T)})$.

$$L_T P(z_t = i | X^{(T)} = x^{(T)}) = \begin{cases} \alpha_T \Gamma^{t-T}(, i) & t > T \text{ prediction / forecast} \\ \alpha_T(i) & t = T \qquad \text{filtering} \\ \alpha_t(i)\beta_t(i) & 1 \leq t < T \qquad \text{smoothing} \end{cases}$$

$\Gamma^{t-T}(, i)$ refers to the i$^t h$ column of Γ^{t-T}. Also see one of the tutorial questions.

Forecast / Predictive Distribution

$$\begin{aligned} P\left[X_{T+h} = x | X^{(T)} = x^{(T)}\right] &= \frac{P\left[X_{T+h} = x, X^{(T)} = x^{(T)}\right]}{P\left[X^{(T)} = x^{(T)}\right]} \\ &= \frac{\delta P(x_1) \dots \Gamma P(x_T) \Gamma^h P(x) \mathbf{1}'}{L_T} \\ &= \frac{\alpha_T \Gamma^h P(x) \mathbf{1}'}{L_T} \end{aligned}$$

The forecast distribution can be used to construct forecasts and intervals for these forecasts (interval forecasts).

Local Decoding

$$P[C_t = i|X^{(T)} = x^{(T)}] = \frac{P[z_t = i, X^{(T)} = x^{(T)}]}{P[X^{(T)} = x^{(T)}]}$$
$$= \frac{\alpha_t(i)\beta_t(i)}{L_T}$$

For each $t \in \{1, \ldots, T\}$ find the state $i_t^* = argmax(i = 1, \ldots, m)P(C_t = i|X^{(T)} = x^{(T)})$

Global Decoding

Find the sequence $c_1, \ldots, c_T$ which maximises

$$P(C^{(T)} = c^{(T)}|X^{(T)} = x^{(T)})$$

or equivalently, and more conveniently, maximises the joint probability

Why are they equivalent?

$$P(C^{(T)} = C^{(T)}, X^{(T)} = x^{(T)}) = \delta \prod_{t=2}^{T} \gamma_{c_{t-1},c_t} \prod_{t=1}^{T} p_{c_t}(x_t)$$

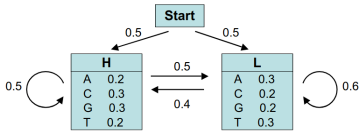
over all possible state sequences. There are m^T possible sequences, which is prohibitive for large T.

Genetic Example

Consider the following simple HMM. This model is composed of 2 states, H (high GC content) and L (low GC content). We can, for example, consider that state H characterizes coding DNA while L characterizes a non-coding DNA. The model can then be used to predict the region of coding DNA from a given sequence.

Suppose we have observed the sequence `GGCACTGAA`.

`LLHHHHLLL` is one possible path of states. We can calculate the probability of this path as



State transition diagram for the genetic example. Source: https://personal.utdallas.edu/~prr105020/biol6385/2017/lecture/Viterbi_handout.pdf

$$P(C^{(T)} = LLHHHHLLL) = \delta_L \times \gamma_{LL} \times \gamma_{LH} \ldots$$
$$= 0.5 \times 0.6 \times 0.4 \times 0.5 \times 0.5 \times 0.5 \times 0.5 \times 0.6^2$$
$$= 0.0027$$

The probabilities of all of these 2^9 possible paths would add up to 1. However, we need the joint probabilities of the path AND the observed data. Again the probabilities of all possible path times observation combinations would add up to 1. We are however only interested in a single observation sequence.

$$\begin{aligned} P(C^{(T)} = LLHHHHLLL) &= \delta_L \times p_L(G) \times \gamma_{LL} \times p_L(G) \times \gamma_{LH} \dots \\ &= 0.5 \times 0.2 \times 0.6 \times 0.2 \times 0.4 \times 0.3 \dots ... \end{aligned}$$

We are interested in maximising, over all possible paths $z^{(T)} = z_1, \dots, z_T$

$$P(C^{(T)} = c^{(T)}, X^{(T)} = x^{(T)}).$$

Instead of looking at all possible paths ($O(m^T)$) we can use a dynamic programming algorithm called the **Viterbi algorithm**.

Viterbi Algorithm

Define

$$\xi_{1i} = P(C_1 = i, X_1 = x_1) = \delta_i p_i(x_1).$$

ξ = xi

Then, for $t = 2, \dots, T$

$$\xi_{ti} = \max_{c_1,\dots,c_{t-1}} P(C^{(t-1)} = c^{(t-1)}, C_t = i, X^{(T)} = x^{(T)}).$$

ξti is the maximum probability of being in state i at time t, maximised over all possible previous paths leading to state i at time t, given all of the observations.

The most probable path to state i at time t must consist of the most probable path to state j at time $t-1$, followed by a transition from j to i. Therefore we can calculate the probabilities ξ_{tj} recursively:

$$\xi_{tj} = \left[\max_i(\xi_{t-1,i} \times \gamma_{ij})\right] p_j(x_t).$$

This implies that, if we have the most likely path to state i at time t (defined by ξ_{ti}, we can find the most likely path to state j at time $t+1$ by maximising over all $\xi_t i \times \gamma_{ij}$. The new $\xi_{t+1,j}$ is found by multiplying this maximum path by the probability of observing x_t from state j.

	G		G		C	A	C	T	G	A
H	-1 -1.737 = -2.737	- 1 = **-3.737**	-1.737 = -5.474	-1 = **-6.474**	-1.737 = -8.211					
L-H		-1.322 = -4.644		-1.322 = -7.381						
H-L		-1 = **-3.737**		-1 = **-6.474**						
L	-1 -2.322 = -3.322	-0.737 = -4.059	-2.322 = -6.059	-0.737 = -6.796	-2.322 = -8.796					

Here we are maximizing over all possible states, the $\xi_{t-1,i}$ capturing the most probable path up to this state at time $t-1$.

ξ_{tj} can be stored in a $T \times m$ matrix. The Viterbi algorithm uses of the order $O(Tm)$ steps, it is linear in T, MUCH better than the $O(m^T)$ required when having to list all possible joint probabilities of states and observations.

Once we have calculated all of the ξ_{ti} we need to find the *maximising sequence*, call this $S_1, \ldots, S_T$.

We start at the end:

$$S_T = \underset{i=1,\ldots,m}{\text{argmax}} \xi_{Ti}.$$

Then, for $t = T-1, \ldots, 1$:

$$S_t = \underset{i=1,\ldots,m}{\text{argmax}} (\xi_{ti} \gamma_{i,S_{t+1}})$$

The calculations for ξ_{ti} and the Viterbi algorithm involve products of probabilities. To avoid underflow, log-probabilities are used.

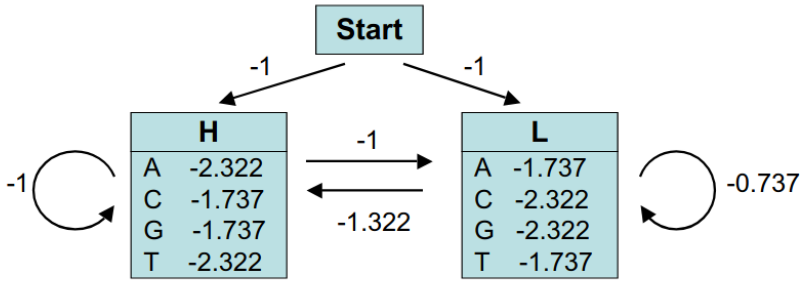


Figure 9: State transition diagram for the genetic example with log-probabilities, using log base 2.

Table 1 illustrates the first few steps of the Viterbi algorithm for the genetic example. The top row shows the observations, the four rows below the four possible state transitions. Columns with observation give ξ_{ti}, columns in between show $\xi_{t-1,i}\gamma_{ij}$, with maximum underlined. Log-probabilities (base 2) are added together.

By hand this is way too much work. The following R program shows an implementation of the Viterbi algorithm for the genetic example above.

```
n <- 9

## emission probabilities
em.probs <- matrix(c(0.2, 0.3, 0.3, 0.2,
                     0.3, 0.2, 0.2, 0.3),
                   byrow = TRUE, nrow = 2)
em.probs <- log2(em.probs)

## observations
obs <- c("G","G","C","A","C","T","G","A","A")

## possible observations
obsposs <- c("A", "C", "G", "T")

init <- c(0.5, 0.5)
init <- log2(init)

Gamma <- matrix(c(0.5, 0.5, 0.4, 0.6),
                byrow = TRUE, nrow = 2)
Gamma <- log2(Gamma)

xi <- matrix(0, nrow = 2, ncol = n)
xi[,1] <- init + em.probs[,3]

for (i in 2:n) {
  oi <- which(obs[i] == obsposs)
 for (j in 1:2) {
   xi[j,i] <- max(xi[,i-1] + Gamma[,j]) +
     em.probs[j,oi]
 }
}

xi
```

```
          [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
[1,] -2.736966 -5.473931 -8.210897 -11.53282 -14.00676 -17.32868 -19.53958
[2,] -3.321928 -6.058894 -8.795859 -10.94786 -14.00676 -16.48069 -19.53958
          [,8]      [,9]
[1,] -22.86151 -25.65737
[2,] -22.01351 -24.48744
```

As a last step we need to do back-tracing: find the most likely path. At each time point, starting from the end, select the most likely previous state that led up to the current state.

Here is some pseudo-code for the back-tracing step:

1. S_T: which state i maxizes ξ_{Ti}
2. for (t in T-1:1)

- S_t: which state i maxizes $\xi_{ti}\gamma_{i,S_{t-1}}$
- This involves element-wise multiplication of columns $\xi_{.t}$ and $\gamma_{S_{t-1}}$.

Example: Earthquakes

```
eq <- read.table("earthquakes.txt")
names(eq) <- c("year", "quakes")
#quakes <- ts(eq$quakes, start = 1900)

library(depmixS4)

mod2 <- depmix(quakes ~ 1, data = eq, nstates = 2, family = poisson())
mod1 <- depmix(quakes ~ 1, data = eq, nstates = 1, family = poisson())
mod3<- depmix(quakes ~ 1, data = eq, nstates = 3, family = poisson(),
              respstart = c(10, 15, 20))

fm2 <- fit(mod2)
```

```
converged at iteration 28 with logLik: -341.8787
```

```
fm1 <- fit(mod1)
```

```
converged at iteration 1 with logLik: -391.9189
```

```
fm3 <- fit(mod3)
```

```
converged at iteration 24 with logLik: -328.5275
```

```
AIC(fm1, fm2, fm3)
```

```
    df      AIC
fm1  1 785.8379
fm2  5 693.7574
fm3 11 679.0550
```

```
(fit3 <- summary(fm3))
```

```
Initial state probabilities model
pr1 pr2 pr3
  0   1   0

Transition matrix
        toS1  toS2  toS3
fromS1 0.906 0.040 0.053
fromS2 0.032 0.939 0.029
fromS3 0.190 0.000 0.810

Response parameters
Resp 1 : poisson
    Re1.(Intercept)
St1           2.981
St2           2.575
St3           3.392


    Re1.(Intercept)
St1           2.981
St2           2.575
St3           3.392
```

```r
# posterior:
# the first column has the Viterbi states,
# the other columns have the xi probabilities (delta in Rabiner), see Rabiner (1989).

post <- posterior(fm3)
head(post)
```

```
  state          S1        S2           S3
1     2 0.000000000 1.0000000 0.000000e+00
2     2 0.013797029 0.9860287 1.743174e-04
3     2 0.001222184 0.9987765 1.317557e-06
4     2 0.002749238 0.9972440 6.732132e-06
5     2 0.030533643 0.9685901 8.762765e-04
6     3 0.304694686 0.1665380 5.287673e-01
```

```r
# apply(post[,2:3], 1, sum)
# sum(post[,2:3])

vit <- (post[,1])

means <- exp(fit3[,1])
state.est <- means[vit]

plot(quakes, las = 1, type = "b", pch = 19, cex = 0.5)
lines(c(time(quakes)), state.est, lwd = 2, col = "red")
```

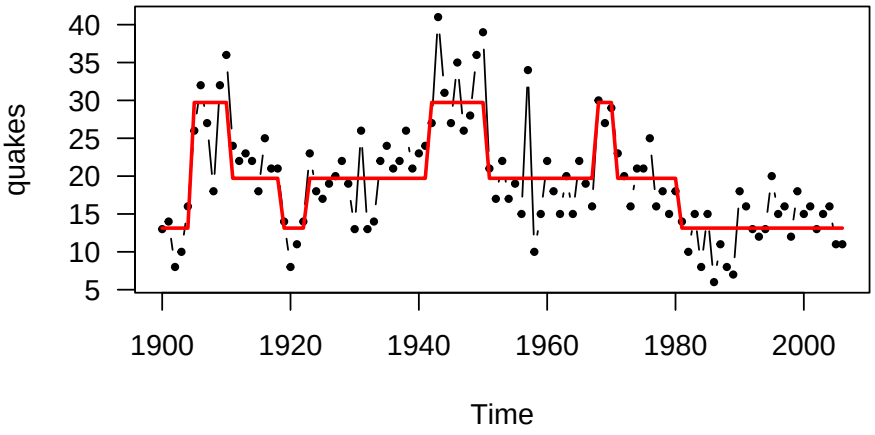


Figure 10: Most probable sequence of states applied to earth quakes data, using Viterbi algorithm.

The state parameters λ_i seem to be estimated on the log-scale, but backtransformed the means of the three Poisson distributions are 19.7, 13.1, 29.7.

Exercises

1. Consider a stationary two-state Poisson-HMM with parameters

$$\Gamma = \begin{pmatrix} 0.1 & 0.9 \\ 0.4 & 0.6 \end{pmatrix}$$

and $\lambda = (1, 3)$.

In each of the following ways, compute the probability that the first three observations from this model are 0, 2, 1.

a. Consider all possible sequences of states of the Markov chain that could have occurred. Compute the probability of each sequence, and the probability of the observations given each sequence.

b. Apply the formula

$$Pr(X_1 = 0, X_2 = 2, X_3 = 1) = \delta \mathbf{P}(0)\Gamma\mathbf{P}(2)\Gamma\mathbf{P}(1)\mathbf{1}'.$$

2. We have the general expression

$$L_T = \delta \mathbf{P}(x_1)\Gamma\mathbf{P}(x_2)\ldots\Gamma\mathbf{P}(x_T)\mathbf{1}'$$

for the likelihood of an HMM, e.g. of Poisson type. Consider the special case in which the Markov chain degenerates to a sequence of independent random variables, i.e. an independent mixture model.

Show that, in this case, the likelihood simplifies to

$$L_T(\theta_1, ...\theta_m, \delta_1, \ldots, \delta_m) = \prod_{j=1}^{n} \sum_{i=1}^{m} \delta_i p_i(x_j, \theta_i)$$

3. How would you choose the number of states to use?
4. What are the parameters in the following HMM?
5. Describe the situation for which you would typically choose a HMM.
6. Show that

$$\alpha_t = P(x_1, \ldots, x_t, z_t)$$

can be found recursively as

$$\alpha_t = P(x_t|z_t)\alpha_{t-1}P(z_t|z_{t-1}) = \alpha_{t-1}\Gamma P(x_t).$$

7. Show that $\alpha_1 \mathbf{1}' = L_1$, and more generally $\alpha_t \mathbf{1}' = L_t$.
8. Here is a simple Bernoulli HMM. The initial distribution and transition probability matrix are as follows:

$$\delta = \begin{pmatrix} \frac{1}{3} & \frac{2}{3} \end{pmatrix} \qquad \Gamma = \begin{pmatrix} 0.5 & 0.5 \\ 0.25 & 0.75 \end{pmatrix}$$

Possible states: 1, 2

Under state 1: $P(0) = P(1) = 0.5$

Under state 2: $P(1) = 1$

Assume you have observed the sequence $x_1 = 1, x_2 = 1$.

a. Find α_2, the forward probabilities at time 2. Interpret.
b. Find the likelihood.
c. Find the predictive / forecast distribution for x_3. Interpret.
d. Find the most likely sequence of states, given the observations.
e. For each time $t = 1, \ldots, 2$, find the most likely state, given the observations.
f. Predict the state at time 3, $Pr(z_3 = i|x_1, x_2)$.

9. Prove

$$P(z_{T+1} = i|X^{(T)} = x^{(T)}) = \alpha_T \Gamma(, i)/L_T.$$

Maths Extras

Product rule:

$$p(x_1, \ldots, x_N) = \prod_{n=1}^{N} p(x_n | x_1, \ldots, x_{n-1})$$

Acknowledgements

These notes were compiled from mainly 3 sources: Murphy, Bishop, and Zucchini & MacDonald (see references).

References

- Bishop, C.M. (2006) Pattern recognition and machine learning. New York: Springer (Information science and statistics).
- Jurafsky and Martin (2019) Hidden Markov Models in Speech and Language Processing. Available at: https://web.stanford.edu/~jurafsky/slp3/A.pdf.
- Murphy, K.P. (2012) Machine learning: a probabilistic perspective. Cambridge, MA: MIT Press (Adaptive computation and machine learning series).
- Rabiner, L. (1989) 'A tutorial on hidden Markov models and selected applications in speech recognition', Proceedings of the IEEE, 77. Available at: http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=18626.
- Rabiner, L.R. (no date) 'An Introduction to Hidden Markov Models', p. 13.
- Stamp, M. (2017) 'A Revealing Introduction to Hidden Markov Models', in Stamp, M., Introduction to Machine Learning with Applications in Information Security. 1st edn. Chapman and Hall/CRC, pp. 7–35. Available at: https://doi.org/10.1201/9781315213262-2.
- Ingmar Visser, Maarten Speekenbrink (2010). depmixS4: An R Package for Hidden Markov Models. Journal of Statistical Software, 36(7), 1-21. https://www.jstatsoft.org/v36/i07/.
- Visser, I. (2011) 'Seven things to remember about hidden Markov models: A tutorial on Markovian models for time series', Journal of Mathematical Psychology, 55(6), pp. 403–415. Available at: https://doi.org/10.1016/j.jmp.2011.08.002.
- Zucchini, W., MacDonald, I.L., & Langrock, R. (2016). Hidden Markov Models for Time Series: An Introduction Using R, Second Edition (2nd ed.). Chapman and Hall/CRC. https://doi.org/10.1201/b20790

Zucchini, W, and IL MacDonald. 2009. *Hidden Markov Models for Time Series: An Introduction Using R.*